Cryptography

COMP2711 Discrete Mathematical Tools

Mingxun Zhou (mingxunz@ust.hk)

This lecture: Speedrunning 3 Turing Awards?

Today: Diffie–Hellman and RSA.


Diffie


Hellman

1976
Public-key framework
and key exchange.

2015 Turing Award

Photos: DH: Mary Holzer / Martin Hellman; composite Phr (CC BY-SA 3.0); Rivest: Ronald L. Rivest (CC BY-SA 4.0); Shamir: The Royal Society (CC BY-SA 3.0); Adleman: Len Adleman (CC BY-SA 3.0); Goldwasser: Weizmann Institute (public domain); Micali: Rama (CC BY-SA 2.0 FR).

This lecture: Speedrunning 3 Turing Awards?

Today: Diffie–Hellman and RSA.


Diffie


Hellman

1976
Public-key framework and key exchange.

2015 Turing Award


Rivest


Shamir


Adleman

1977: RSA
Public-key encryption and digital signatures.

2002 Turing Award

Photos: DH: Mary Holzer / Martin Hellman; composite Phr (CC BY-SA 3.0); Rivest: Ronald L. Rivest (CC BY-SA 4.0); Shamir: The Royal Society (CC BY-SA 3.0); Adleman: Len Adleman (CC BY-SA 3.0); Goldwasser: Weizmann Institute (public domain); Micali: Rama (CC BY-SA 2.0 FR).

This lecture: Speedrunning 3 Turing Awards?

Today: Diffie–Hellman and RSA.




Diffie Hellman

1976
Public-key framework and key exchange.

2015 Turing Award





Rivest Shamir Adleman

1977: RSA
Public-key encryption and digital signatures.

2002 Turing Award




Goldwasser Micali

1982
Probabilistic encryption and rigorous security.

2012 Turing Award

Photos: DH: Mary Holzer / Martin Hellman; composite Phr (CC BY-SA 3.0); Rivest: Ronald L. Rivest (CC BY-SA 4.0); Shamir: The Royal Society (CC BY-SA 3.0); Adleman: Len Adleman (CC BY-SA 3.0); Goldwasser: Weizmann Institute (public domain); Micali: Rama (CC BY-SA 2.0 FR).

Contents

1. Symmetric Encryption
2. RSA public-key encryption
3. Diffie–Hellman key exchange
4. Optional extensions
5. Review exercises

Private messages over a public channel

Plaintext: the original message Alice wants to send, `MEET AT NOON`.



Private messages over a public channel

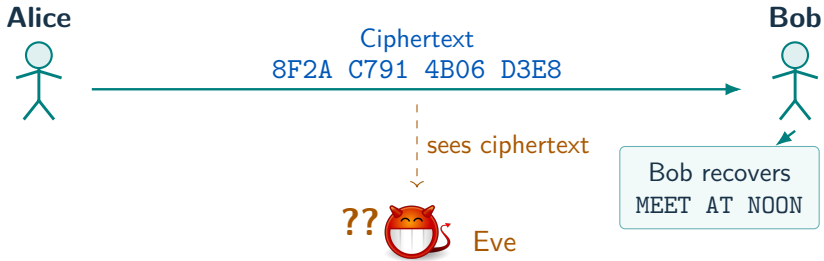
Plaintext: the original message Alice wants to send, `MEET AT NOON`.



Issue: An eavesdropper listening to the public channel can learn the message.

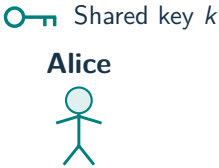
Private messages over a public channel

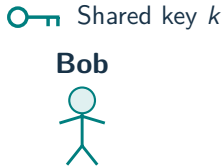
Plaintext: the original message Alice wants to send, `MEET AT NOON`.



Target: Bob recovers the message; Eve cannot understand it.

Symmetric encryption





Symmetric encryption



Alice



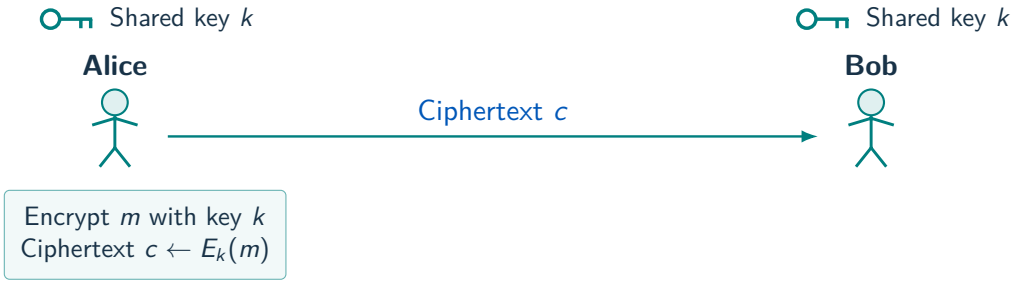
Encrypt m with key k
Ciphertext $c \leftarrow E_k(m)$



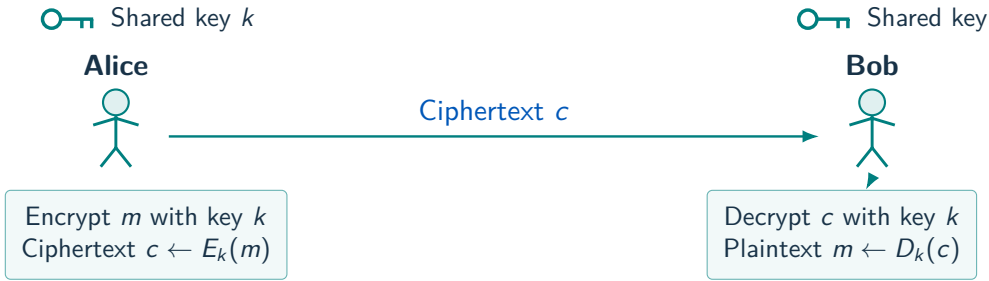
Bob



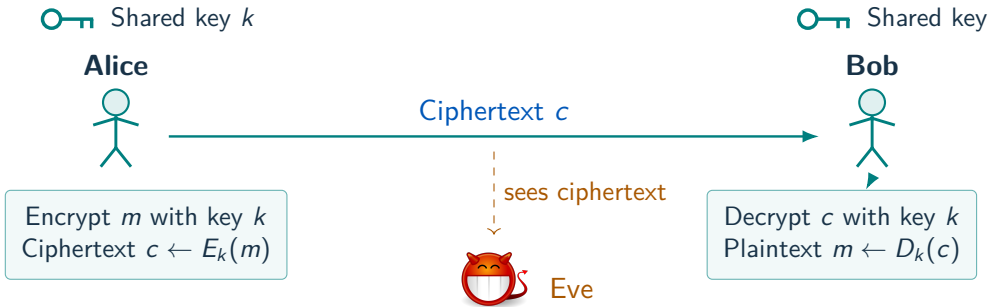
Symmetric encryption



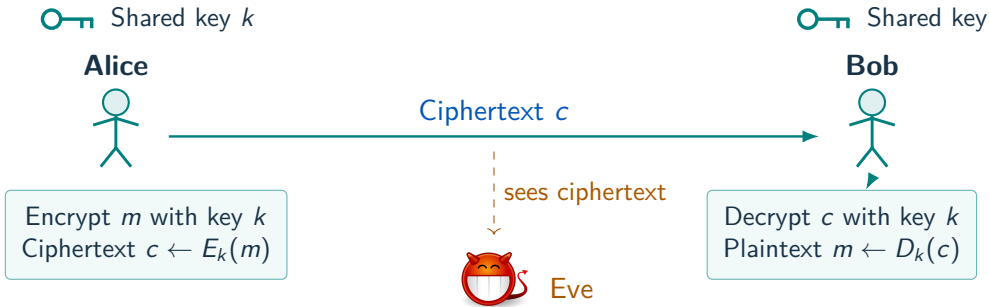
Symmetric encryption



Symmetric encryption



Symmetric encryption

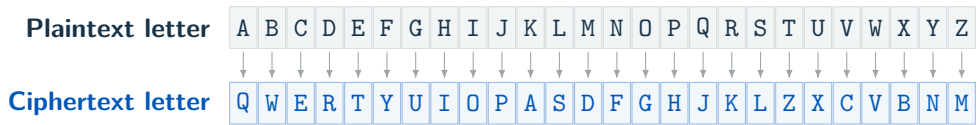


Modern cryptography principle

The encryption/decryption algorithm is **public**. Only the key is private.

Example: A substitution cipher

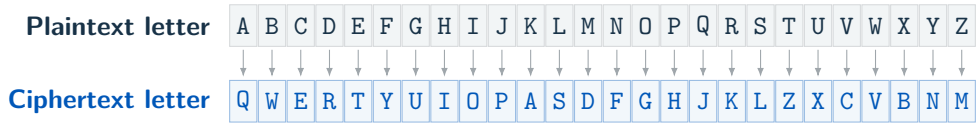
Alice and Bob share the **same substitution table** as their **secret key** k.



Plaintext (original message) `MEET AT NOON`

Example: A substitution cipher

Alice and Bob share the **same substitution table** as their **secret key** k.



Plaintext (original message) `MEET AT NOON`

Ciphertext (encrypted message) `DTTZ QZ FGGF`

From plaintext to ciphertext

Alice encrypts using the secret table; Bob uses the same table in reverse.

Alice
Plaintext
`MEET AT NOON`

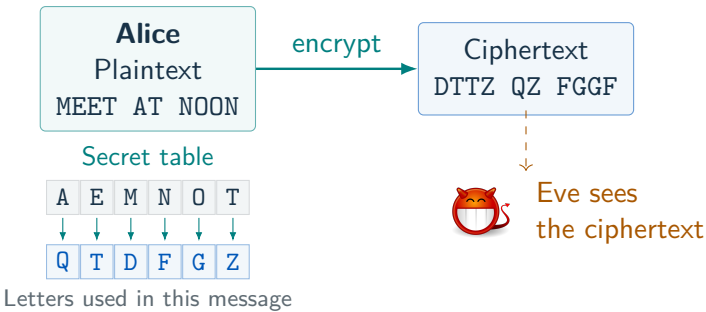
Secret table

A	E	M	N	O	T
↓	↓	↓	↓	↓	↓
Q	T	D	F	G	Z

Letters used in this message

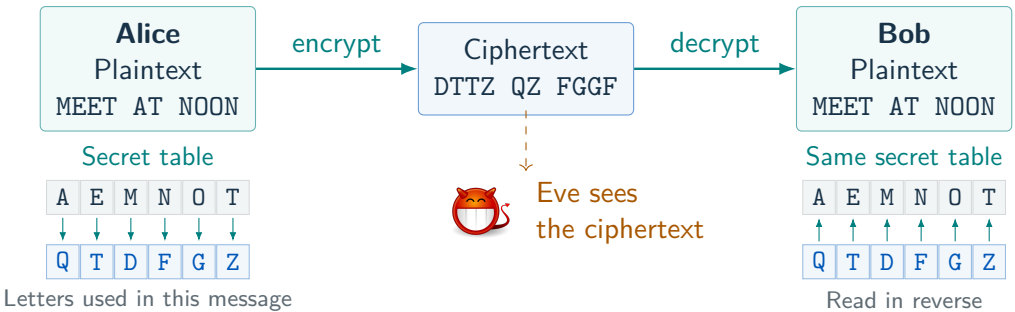
From plaintext to ciphertext

Alice encrypts using the secret table; Bob uses the same table in reverse.



From plaintext to ciphertext

Alice encrypts using the secret table; Bob uses the same table in reverse.



What can Eve infer?

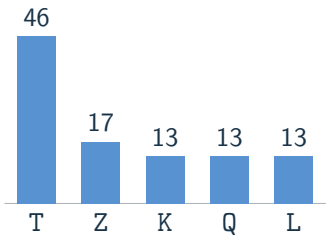
Eve sees the **ciphertext**, but does not know the secret substitution table.



Substitution is not secure: Frequency analysis

A substitution changes the letters, but preserves how often each letter appears.

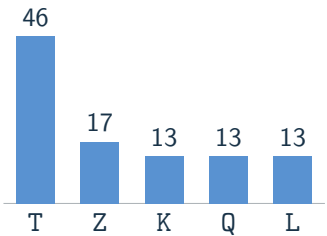
Letter counts in a longer ciphertext



Substitution is not secure: Frequency analysis

A substitution changes the letters, but preserves how often each letter appears.

Letter counts in a longer ciphertext



In English, **E** is usually the most frequent letter.

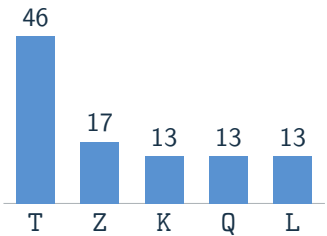
Eve's first guess:

$$\underbrace{\texttt{T}}_{\text{ciphertext}} \longrightarrow \underbrace{\texttt{E}}_{\text{plaintext}}$$

Substitution is not secure: Frequency analysis

A substitution changes the letters, but preserves how often each letter appears.

Letter counts in a longer ciphertext



In English, **E** is usually the most frequent letter.

Eve's first guess:

T (ciphertext) ⟶ E (plaintext)

What other patterns can you think of?

The Enigma machine

Nazi Germany used Enigma, a **substitution-based encryption machine**, to protect its military communications during World War II.



Photo: Robert-brook (CC0). Turing: Elliott & Fry (public domain). History: The Alan Turing Institute.

The Enigma machine

Nazi Germany used Enigma, a **substitution-based encryption machine**, to protect its military communications during World War II.





Alan Turing famously helped crack **Enigma**.

Photo: Robert-brook (CC0). Turing: Elliott & Fry (public domain). History: The Alan Turing Institute.

Your turn: Crack a substitution cipher

Three paragraphs were encrypted using a **new secret substitution table**.

```
yoa ohfi rhfi nfgsaewgyz hp wugafua kfx
yauofhlhiz (ornwy) gw k bnclgu nfgsaewgyz
gf wkg rnfi xgwyeguy, faq yaeegyhegaw,
ohfi rhfi.

...
```

Your task: Recover the message.

https://wuwuz.github.io/COMP2711/Materials/cipher.txt

Modern symmetric encryption: AES

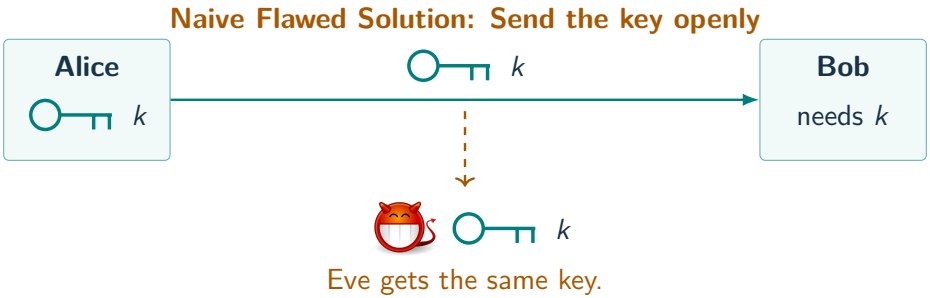
Advanced Encryption Standard (AES)

Standardized by the US National Institute of Standards and Technology (NIST) in 2001: FIPS 197.

The **most widely used symmetric encryption standard** today.

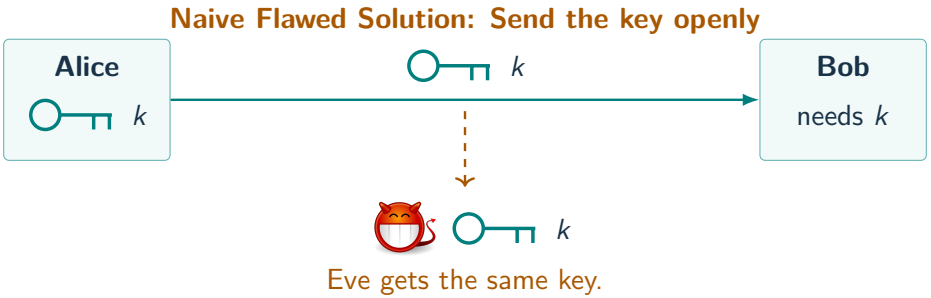
What if they don't share a key upfront?

Both substitution and AES assume Alice and Bob already share a secret key.



What if they don't share a key upfront?

Both substitution and AES assume Alice and Bob already share a secret key.



Can they still communicate securely using only a public channel?

Two approaches

Public-key encryption

Two approaches

Public-key encryption

Key exchange

Contents

1. Symmetric Encryption
2. RSA public-key encryption
3. Diffie–Hellman key exchange
4. Optional extensions
5. Review exercises

Public-key encryption: The setting

Generate $(\mathrm{pk}, \mathrm{sk})$
pk: public key
sk: secret key



Alice



Bob

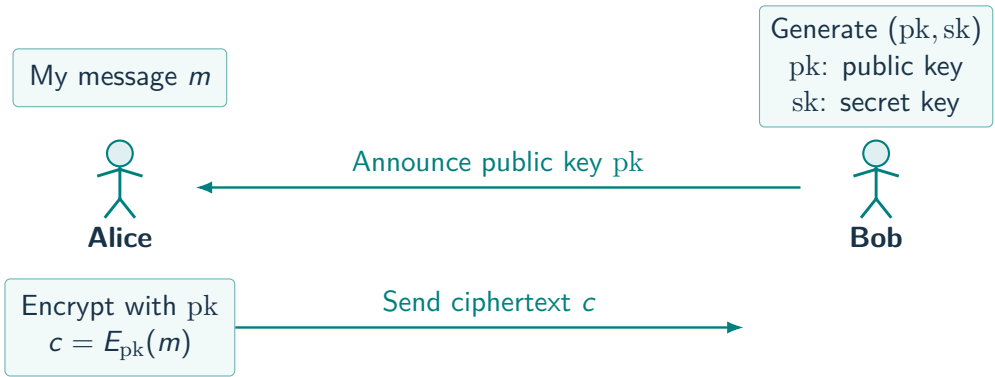
Public-key encryption: The setting



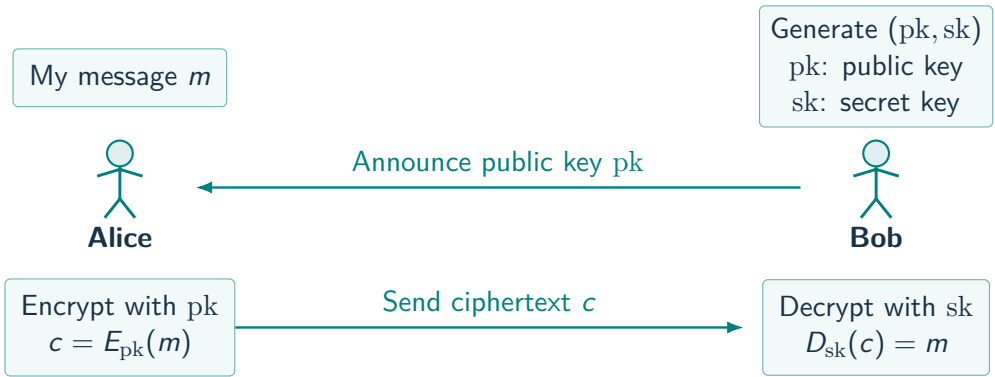
Public-key encryption: The setting



Public-key encryption: The setting



Public-key encryption: The setting



Public-key encryption: The setting



Public-key encryption: The setting



The intuition behind RSA

Goal: Asymmetry

1. **Everyone can encrypt** with the public key pk.
2. **Only Bob can reverse it** with the secret key sk.

The intuition behind RSA

Goal: Asymmetry

1. **Everyone can encrypt** with the public key pk.
2. **Only Bob can reverse it** with the secret key sk.

In modern computers, messages are represented as bits, i.e., integers.

$$\texttt{“Hi”} \longrightarrow \texttt{01001000 01101001} \longrightarrow 18537$$

The intuition behind RSA

Goal: Asymmetry

1. **Everyone can encrypt** with the public key pk.
2. **Only Bob can reverse it** with the secret key sk.

In modern computers, messages are represented as bits, i.e., integers.

$$\texttt{“Hi”} \longrightarrow \texttt{01001000 01101001} \longrightarrow 18537$$

Could we encrypt m using $+$, $-$, or $\times$?

Eve can easily undo addition, subtraction, and invertible multiplication too.

RSA: Try modular exponentiation

Goal: Asymmetry

1. **Everyone can encrypt** with the public key pk.
2. **Only Bob can reverse it** with the secret key sk.

This gives us a strong motivation to try **modular exponentiation** (like a^b mod n).

RSA: Try modular exponentiation

Goal: Asymmetry

1. **Everyone can encrypt** with the public key pk.
2. **Only Bob can reverse it** with the secret key sk.

This gives us a strong motivation to try **modular exponentiation** (like $a^b \bmod n$).
Bob finds n, e, d such that, for every message $0 \leq m < n$,

$$m^{ed} \equiv m \pmod{n}.$$

Let $\mathrm{pk} = (n, e)$ and $\mathrm{sk} = (d)$.

RSA: Try modular exponentiation

Goal: Asymmetry

1. **Everyone can encrypt** with the public key pk.
2. **Only Bob can reverse it** with the secret key sk.

This gives us a strong motivation to try **modular exponentiation** (like $a^b \bmod n$).
Bob finds n, e, d such that, for every message $0 \leq m < n$,

$$m^{ed} \equiv m \pmod{n}.$$

Let $\mathrm{pk} = (n, e)$ and $\mathrm{sk} = (d)$.

The proposed public-key encryption scheme

1. **Everyone can encrypt:** $c = m^e \bmod n$.
2. **Only Bob can decrypt:** $c^d \equiv m^{ed} \equiv m \pmod{n}$.

RSA: Try modular exponentiation

Goal: Asymmetry

1. **Everyone can encrypt** with the public key pk.
2. **Only Bob can reverse it** with the secret key sk.

Remaining challenge

How do we choose suitable n, e, d?

The proposed public-key encryption scheme

1. **Everyone can encrypt:** $c = m^e \bmod n$.
2. **Only Bob can decrypt:** $c^d \equiv m^{ed} \equiv m \pmod{n}$.

Recall: Modular exponentiation

Euler's theorem: Reducing exponents

For $n > 1$, $\gcd(m, n) = 1$, and an integer $k \geq 0$,

$$m^k \equiv m^{k \bmod \varphi(n)} \pmod{n}.$$

Recall: Modular exponentiation

Euler's theorem: Reducing exponents

For $n > 1$, $\gcd(m, n) = 1$, and an integer $k \geq 0$,

$$m^k \equiv m^{k \bmod \varphi(n)} \pmod{n}.$$

Recall: $\varphi(n)$ counts the integers $x \in \{1, \ldots, n-1\}$ with $\gcd(x, n) = 1$.

Recall: Modular exponentiation

Euler's theorem: Reducing exponents

For $n > 1$, $\gcd(m, n) = 1$, and an integer $k \geq 0$,

$$m^k \equiv m^{k \bmod \varphi(n)} \pmod{n}.$$

Recall: $\varphi(n)$ counts the integers $x \in \{1, \ldots, n-1\}$ with $\gcd(x, n) = 1$.

Our remaining goal is to find positive exponents e, d such that

$$ed \equiv 1 \pmod{\varphi(n)}.$$

Recall: Modular exponentiation

Euler's theorem: Reducing exponents

For $n > 1$, $\gcd(m, n) = 1$, and an integer $k \geq 0$,

$$m^k \equiv m^{k \bmod \varphi(n)} \pmod{n}.$$

Recall: $\varphi(n)$ counts the integers $x \in \{1, \ldots, n-1\}$ with $\gcd(x, n) = 1$.

Our remaining goal is to find positive exponents e, d such that

$$ed \equiv 1 \pmod{\varphi(n)}.$$

Then, for $\gcd(m, n) = 1$,

$$m^{ed} \equiv m^{ed \bmod \varphi(n)} \equiv m^1 = m \pmod{n}.$$

RSA: Choosing suitable n, e, d

Recall our security target

Eve should not be able to efficiently recover the secret key $\mathrm{sk} = (d)$ from the public key $\mathrm{pk} = (n, e)$.

She knows that d is the inverse of e modulo $\varphi(n)$:

$$d = e^{-1} \bmod \varphi(n).$$

RSA: Choosing suitable n, e, d

Recall our security target

Eve should not be able to efficiently recover the secret key $\mathrm{sk} = (d)$ from the public key $\mathrm{pk} = (n, e)$.

She knows that d is the inverse of e modulo $\varphi(n)$:

$$d = e^{-1} \bmod \varphi(n).$$

If Eve can compute $\varphi(n)$ efficiently, she can recover d using extended Euclid.

RSA: Choosing suitable n, e, d

Recall our security target

Eve should not be able to efficiently recover the secret key $\mathrm{sk} = (d)$ from the public key $\mathrm{pk} = (n, e)$.

She knows that d is the inverse of e modulo $\varphi(n)$:

$$d = e^{-1} \bmod \varphi(n).$$

If Eve can compute $\varphi(n)$ efficiently, she can recover d using extended Euclid.

The key idea in RSA

Choose n so that **Bob can compute $\varphi(n)$ easily**, but **computing it is hard for Eve**.

RSA: The connection to factoring

Recall: $\varphi(n)$ is closely connected to **factoring**.

For distinct primes p, q

$$n = pq \qquad \Longrightarrow \qquad \varphi(n) = \varphi(p) \times \varphi(q) = (p-1)(q-1).$$

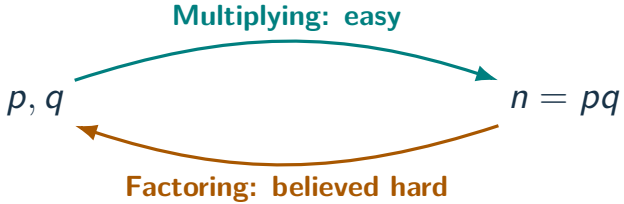
RSA: The connection to factoring

Recall: $\varphi(n)$ is closely connected to **factoring**.

For distinct primes p, q

$$n = pq \qquad \Longrightarrow \qquad \varphi(n) = \varphi(p) \times \varphi(q) = (p-1)(q-1).$$

For suitably chosen large primes, factoring gives us an **asymmetry**:



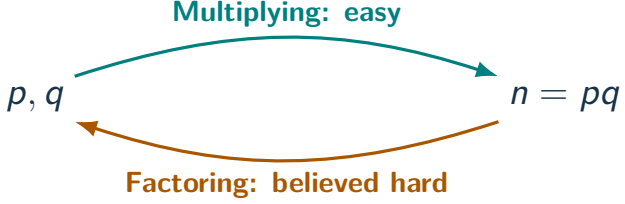
RSA: The connection to factoring

Recall: $\varphi(n)$ is closely connected to **factoring**.

For distinct primes p, q

$$n = pq \qquad \Longrightarrow \qquad \varphi(n) = \varphi(p) \times \varphi(q) = (p-1)(q-1).$$

For suitably chosen large primes, factoring gives us an **asymmetry**:



The core idea in RSA: Bob keeps the factors and publishes their product.

RSA: Give Bob a secret advantage

Bob chooses two distinct large primes p, q, and publishes $n = pq$.

RSA: Give Bob a secret advantage

Bob chooses two distinct large primes p, q, and publishes $n = pq$.

Bob already knows $n = pq$

He knows p, q, so he can compute

$$\varphi(n) = (p-1)(q-1).$$

and the reverse exponent

$$d = e^{-1} \bmod \varphi(n).$$

Eve sees only n and exponent e

- To compute $d = e^{-1} \bmod \varphi(n)$ by this route, she needs $\varphi(n)$.
- Computing $\varphi(n)$ is equivalent to factoring n into $p \times q$.
- **Factoring is believed to be hard!**

RSA: 1. Generate the keys

1. Choose distinct large primes p, q; let $n = pq$.



Alice



Bob

Public: n, e

Private: $p, q, \varphi(n), d$

RSA: 1. Generate the keys

1. Choose distinct large primes p, q; let $n = pq$.
2. Choose e coprime to $\varphi(n) = (p-1)(q-1)$.



Alice



Bob

Public: n, e

Private: $p, q, \varphi(n), d$

RSA: 1. Generate the keys

1. Choose distinct large primes p, q; let $n = pq$.
2. Choose e coprime to $\varphi(n) = (p-1)(q-1)$.
3. $d \equiv e^{-1} \bmod \varphi(n)$ (extended Euclid).



Alice



Bob

Public: n, e

Private: $p, q, \varphi(n), d$

RSA: 2. Encrypt, send, and decrypt

1. Choose distinct large primes p, q; let $n = pq$.
2. Choose e coprime to $\varphi(n) = (p-1)(q-1)$.
3. $d \equiv e^{-1} \bmod \varphi(n)$ (extended Euclid).



Alice

Announce $\mathrm{pk} = (n, e)$



Bob

Public: n, e

Private: $p, q, \varphi(n), d$

RSA: 2. Encrypt, send, and decrypt

1. Choose distinct large primes p, q; let $n = pq$.
2. Choose e coprime to $\varphi(n) = (p-1)(q-1)$.
3. $d \equiv e^{-1} \bmod \varphi(n)$ (extended Euclid).



Alice

Announce $\mathrm{pk} = (n, e)$



Bob

Public: n, e
Private: $p, q, \varphi(n), d$

Message $0 \le m < n$
Encrypt: $c = m^e \bmod n$

RSA: 2. Encrypt, send, and decrypt

1. Choose distinct large primes p, q; let $n = pq$.
2. Choose e coprime to $\varphi(n) = (p-1)(q-1)$.
3. $d \equiv e^{-1} \bmod \varphi(n)$ (extended Euclid).



Alice

Announce $\mathrm{pk} = (n, e)$



Bob

Public: n, e
Private: $p, q, \varphi(n), d$

Message $0 \le m < n$
Encrypt: $c = m^e \bmod n$

Send ciphertext c

RSA: 2. Encrypt, send, and decrypt

1. Choose distinct large primes p, q; let $n = pq$.
2. Choose e coprime to $\varphi(n) = (p-1)(q-1)$.
3. $d \equiv e^{-1} \bmod \varphi(n)$ (extended Euclid).



Alice

Announce $\mathrm{pk} = (n, e)$



Bob

Public: n, e
Private: $p, q, \varphi(n), d$

Message $0 \le m < n$
Encrypt: $c = m^e \bmod n$

Send ciphertext c

Decrypt with $\mathrm{sk} = (d)$
$m = c^d \bmod n$

RSA: 2. Encrypt, send, and decrypt

1. Choose distinct large primes p, q; let $n = pq$.
2. Choose e coprime to $\varphi(n) = (p-1)(q-1)$.
3. $d \equiv e^{-1} \bmod \varphi(n)$ (extended Euclid).



Alice

Announce $\mathrm{pk} = (n, e)$



Bob

Public: n, e
Private: $p, q, \varphi(n), d$

Message $0 \le m < n$
Encrypt: $c = m^e \bmod n$

Send ciphertext c

Decrypt with $\mathrm{sk} = (d)$
$m = c^d \bmod n$



Eve's view: (n, e) and ciphertext c

Textbook RSA is not secure

$$c = m^e \bmod n.$$

Deterministic: the same message always gives the same ciphertext.

Textbook RSA is not secure

$$c = m^e \bmod n.$$

Deterministic: the same message always gives the same ciphertext.

Exercise: What happens if Alice always encodes "Yes" as $m = 1$ and "No" as $m = 0$?

Textbook RSA is not secure

$$c = m^e \bmod n.$$

Deterministic: the same message always gives the same ciphertext.

Exercise: What happens if Alice always encodes "Yes" as $m = 1$ and "No" as $m = 0$?

In practice

Use randomized encoding, such as **RSA-OAEP**, rather than textbook RSA.

Read the course note for the attack and further explanation.

Contents

1. Symmetric Encryption

2. RSA public-key encryption

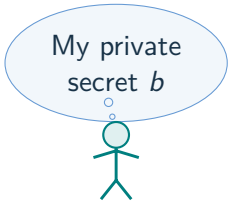
3. Diffie–Hellman key exchange

4. Optional extensions

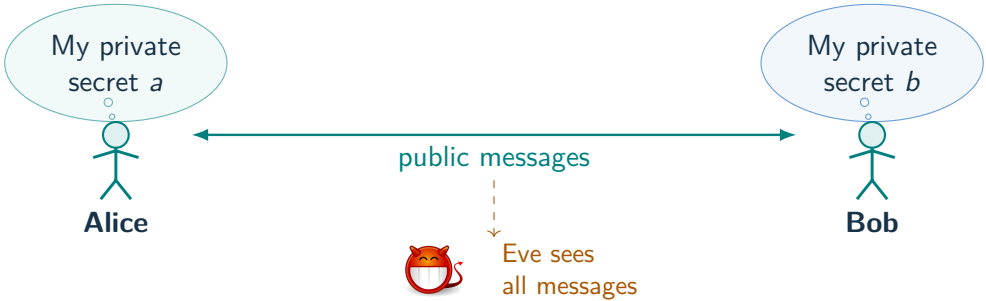
5. Review exercises

Key exchange: The setting

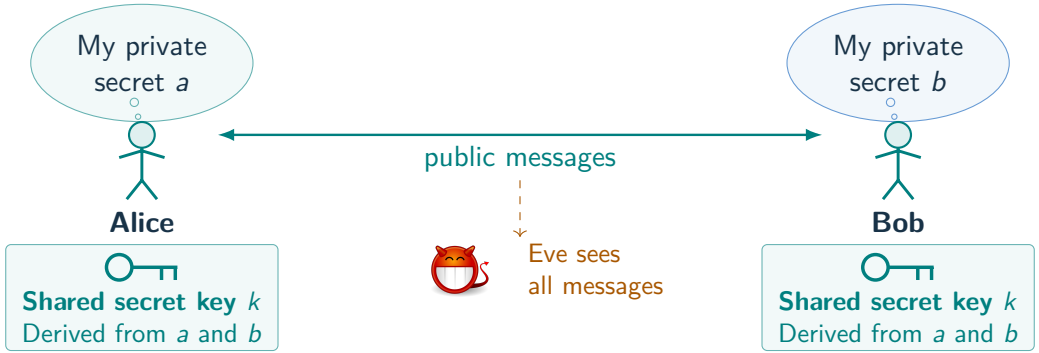




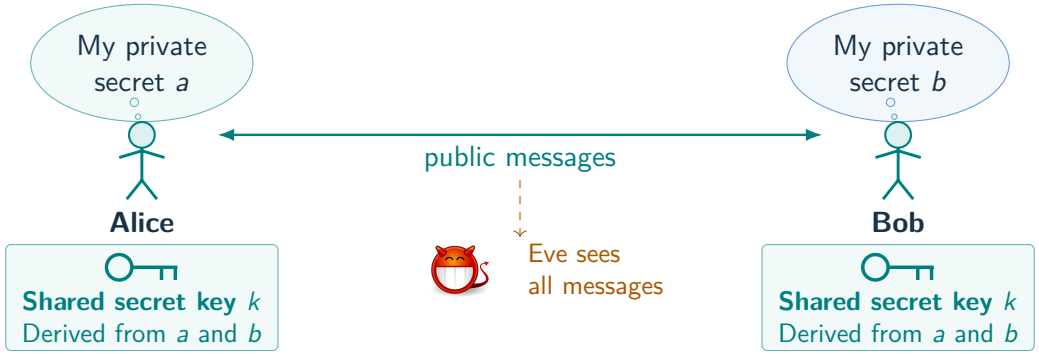
Key exchange: The setting



Key exchange: The setting

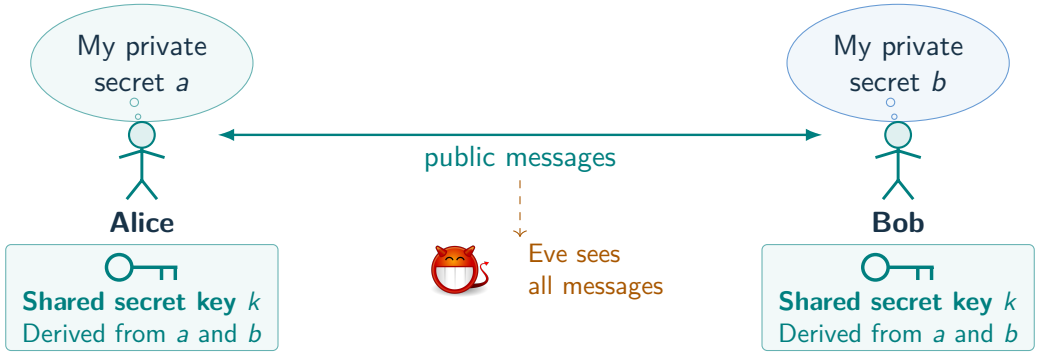


Key exchange: The setting



Security: Eve cannot efficiently recover k.

Key exchange: The setting



Security: Eve cannot efficiently recover k.

Alice and Bob can then use k for symmetric encryption.

Preliminary: Hardness of Discrete Logarithms

We know how to compute:

$$5^6 \bmod 23 = 8.$$

Efficient by repeated squaring.

Preliminary: Hardness of Discrete Logarithms

We know how to compute:

$$5^6 \bmod 23 = 8.$$

What if the exponent is unknown?

$$5^{?} \equiv 8 \pmod{23}.$$

Efficient by repeated squaring.

The discrete logarithm problem

Given a prime p and integers $g, A \in \{1, \ldots, p-1\}$,

$$\text{find an exponent } a \geq 0 \text{ such that } g^a \equiv A \pmod{p},$$

assuming a solution exists.

Preliminary: Hardness of Discrete Logarithms

The discrete logarithm problem

Given a prime p and integers $g, A \in \{1, \ldots, p-1\}$,

$$\text{find an exponent } a \geq 0 \text{ such that } g^a \equiv A \pmod{p},$$

assuming a solution exists.

For suitably chosen large parameters:

Preliminary: Hardness of Discrete Logarithms

The discrete logarithm problem

Given a prime p and integers $g, A \in \{1, \ldots, p-1\}$,

$$\text{find an exponent } a \geq 0 \text{ such that } g^a \equiv A \pmod{p},$$

assuming a solution exists.

For suitably chosen large parameters:

Currently: No efficient classical algorithm is **known** for DLog.

Preliminary: Hardness of Discrete Logarithms

The discrete logarithm problem

Given a prime p and integers $g, A \in \{1, \ldots, p-1\}$,

$$\text{find an exponent } a \geq 0 \text{ such that } g^a \equiv A \pmod{p},$$

assuming a solution exists.

For suitably chosen large parameters:

Currently: No efficient classical algorithm is **known** for DLog.

DLog Assumption

Assume that no efficient classical algorithm for DLog **exists**.

Diffie–Hellman: Key Exchange from Discrete Logarithms

Public: prime p and base $g \not\equiv 0 \pmod{p}$.

Alice: secret a

Bob: secret b

Diffie–Hellman: Key Exchange from Discrete Logarithms

Public: prime p and base $g \not\equiv 0 \pmod{p}$.

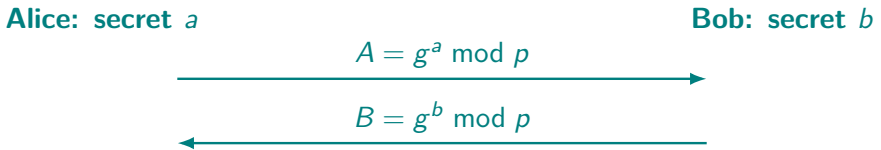
Alice: secret a **Bob: secret** b

$$A = g^a \bmod p$$



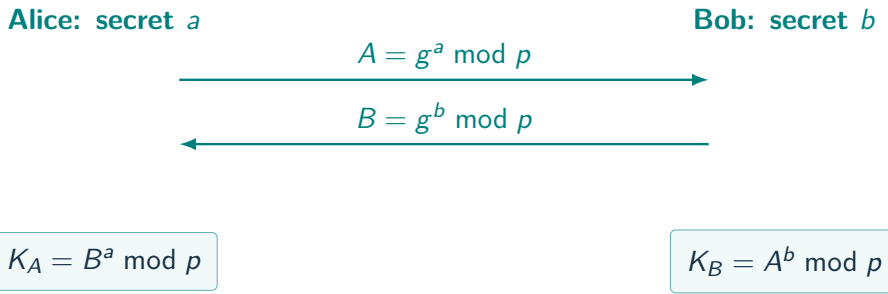
Diffie–Hellman: Key Exchange from Discrete Logarithms

Public: prime p and base $g \not\equiv 0 \pmod{p}$.



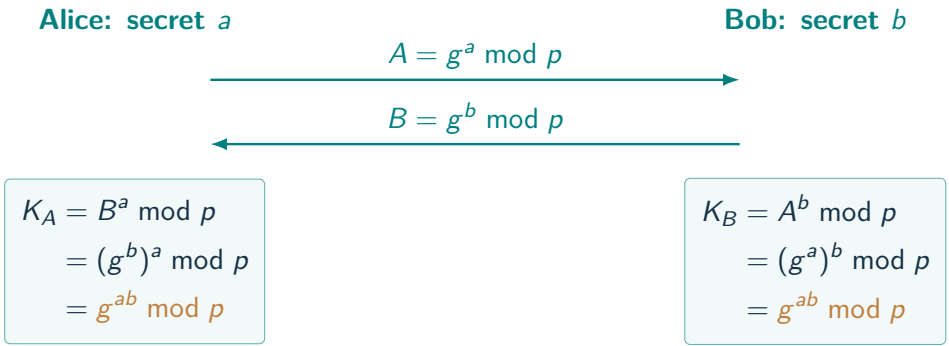
Diffie–Hellman: Key Exchange from Discrete Logarithms

Public: prime p and base $g \not\equiv 0 \pmod{p}$.



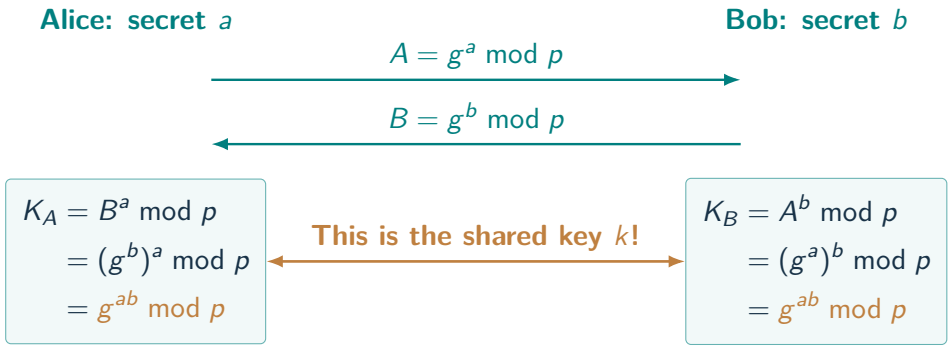
Diffie–Hellman: Key Exchange from Discrete Logarithms

Public: prime p and base $g \not\equiv 0 \pmod{p}$.



Diffie–Hellman: Key Exchange from Discrete Logarithms

Public: prime p and base $g \not\equiv 0 \pmod{p}$.



Diffie–Hellman: A numerical example

Public: $p = 23$ and $g = 5$.

Alice: secret $a = 6$

Bob: secret $b = 15$

Diffie–Hellman: A numerical example

Public: $p = 23$ and $g = 5$.

Alice: secret $a = 6$ **Bob: secret** $b = 15$

$A = 5^6 \bmod 23 = \mathbf{8}$



Diffie–Hellman: A numerical example

Public: $p = 23$ and $g = 5$.

Alice: secret $a = 6$ **Bob: secret** $b = 15$

$A = 5^6 \bmod 23 = \mathbf{8}$ →

← $B = 5^{15} \bmod 23 = \mathbf{19}$

Diffie–Hellman: A numerical example

Public: $p = 23$ and $g = 5$.

Alice: secret $a = 6$ **Bob: secret** $b = 15$

$A = 5^6 \bmod 23 = \mathbf{8}$ →

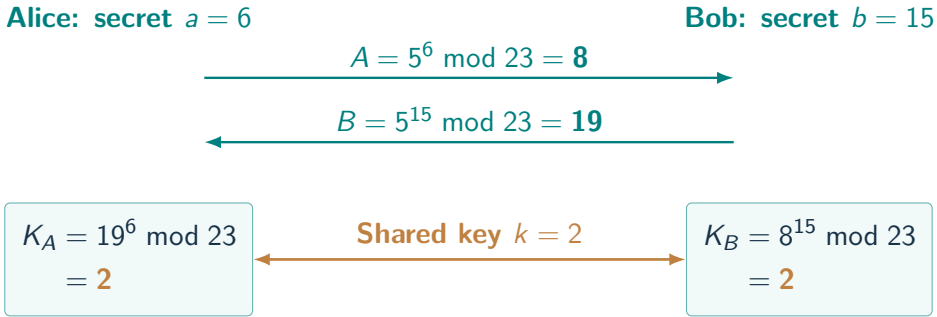
← $B = 5^{15} \bmod 23 = \mathbf{19}$

$K_A = 19^6 \bmod 23$
$= \mathbf{2}$

$K_B = 8^{15} \bmod 23$
$= \mathbf{2}$

Diffie–Hellman: A numerical example

Public: $p = 23$ and $g = 5$.



Security analysis of DH Key Exchange



Security analysis of DH Key Exchange



An intuitive explanation

Assume Discrete Logarithm is Hard.

Eve cannot efficiently recover a from $A = g^a$, or b from $B = g^b$.

Security analysis of DH Key Exchange



An intuitive explanation

Assume Discrete Logarithm is Hard.

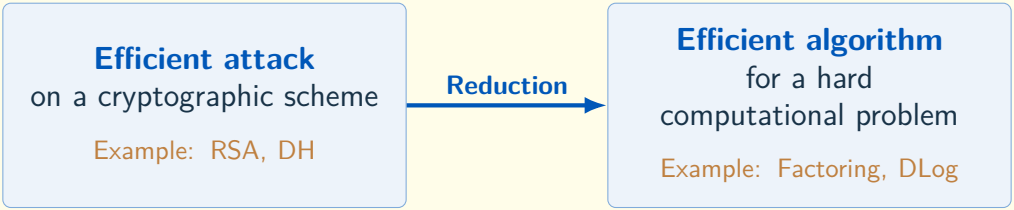
Eve cannot efficiently recover a from $A = g^a$, or b from $B = g^b$.

Intuitively, Eve has no good way to compute $K = g^{ab}$.[1]

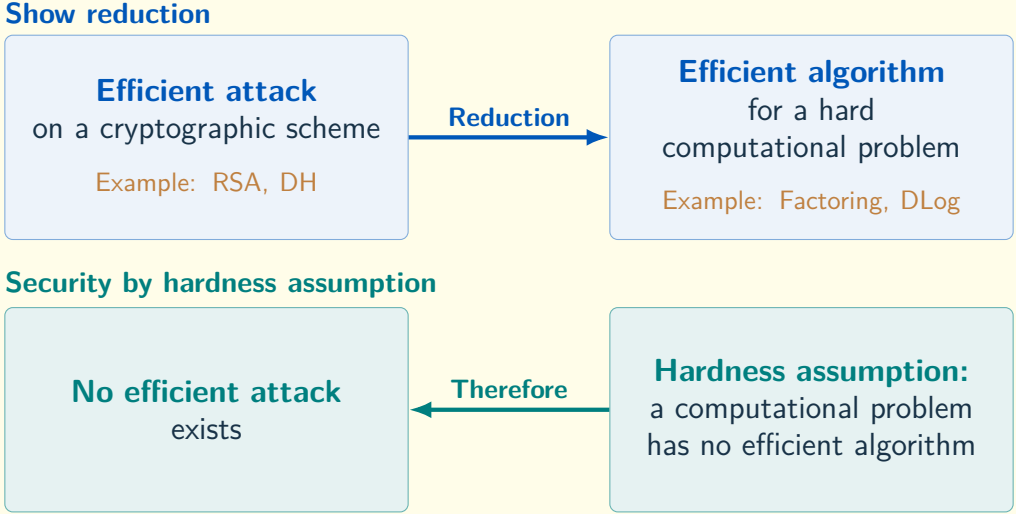
[1] Strictly speaking, key-recovery security requires the stronger Computational Diffie–Hellman (CDH) assumption.

The art of modern cryptography: Security by Reduction

Show reduction



The art of modern cryptography: Security by Reduction



Summary

1. **Symmetric encryption**
   A shared secret key for encryption and decryption.

2. **RSA public-key encryption**
   Public encryption key; private decryption key.

3. **Diffie–Hellman key exchange**
   Establish shared secret material through public messages.

Contents

1. Symmetric Encryption

2. RSA public-key encryption

3. Diffie–Hellman key exchange

4. Optional extensions

5. Review exercises

Optional reading: A numerical RSA example

Key generation

$$p = 5, \quad q = 11, \quad n = 55, \quad \varphi(n) = 40.$$

Choose $e = 3$; its inverse modulo 40 is $d = 27$ since $3 \times 27 = 81 \equiv 1 \pmod{40}$.

$$\mathrm{pk} = (55, 3), \qquad \mathrm{sk} = (27).$$

Send the message $m = 7$.

Alice encrypts: $c = 7^3 \bmod 55 = 13.$

Bob decrypts: $m = 13^{27} \bmod 55 = 7.$

Optional: Computing versus recognizing

Use a suitable prime-order cyclic group: $G = \{1, g, \ldots, g^{r-1}\}$, with $g^r = 1$.

Choose a, b, z independently and uniformly from $\{0, \ldots, r-1\}$.

Optional: Computing versus recognizing

Use a suitable prime-order cyclic group: $G = \{1, g, \ldots, g^{r-1}\}$, with $g^r = 1$.
Choose a, b, z independently and uniformly from $\{0, \ldots, r-1\}$.

CDH: Compute

Given g^a, g^b, compute g^{ab}.

DDH: Distinguish

Tell the two distributions apart:

$$(g^a, g^b, g^{ab})$$

$$(g^a, g^b, g^z).$$

Optional: RSA signatures

Goal: authenticate a message, not hide it.

Let h represent an encoded message digest.

Sign with the private key

$$s = h^d \bmod n.$$

Optional: RSA signatures

Goal: authenticate a message, not hide it.

Let h represent an encoded message digest.

Sign with the private key

$$s = h^d \bmod n.$$

Verify with the public key

$$s^e \bmod n = h.$$

Check the required encoding.

This is the algebraic outline; RSA-PSS supplies a secure signature encoding.

Optional: Multiplying ciphertexts

For textbook RSA, $E(m) = m^e \bmod n$.

Optional: Multiplying ciphertexts

For textbook RSA, $E(m) = m^e \bmod n$.

$$E(m_1)E(m_2) \equiv (m_1 m_2)^e \equiv E(m_1 m_2 \bmod n) \pmod{n}.$$

$$2 \times 3 = 6 \xrightarrow{\text{encrypt with } (55,3)} 8 \times 27 \equiv 51 \pmod{55}$$

Multiplying ciphertexts multiplies the recovered messages modulo n.

Contents

1. Symmetric Encryption

2. RSA public-key encryption

3. Diffie–Hellman key exchange

4. Optional extensions

5. Review exercises

Review exercises: Keys and exchange

1. Key roles. Alice wants to send a private message to Bob. Which key does each party use in symmetric encryption? In public-key encryption?

2. Diffie–Hellman. Let $p = 23$, $g = 5$, $a = 8$, and $b = 5$.

- Find both public values and the shared value.
- List what Eve observes.

Review exercises: RSA

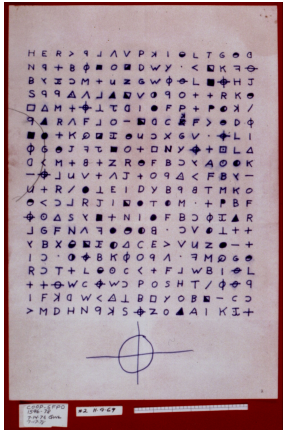
3. RSA. Use $p = 5$, $q = 11$, $e = 3$ as in the optional example.

- Encrypt and decrypt $m = 5$.
- Does Euler's theorem apply directly to this m, n?

4. Challenge. Suppose $n = pq$ for distinct primes. If you know both n and $\varphi(n)$, can you recover p, q?

Try $n = 55$ and $\varphi(n) = 40$.

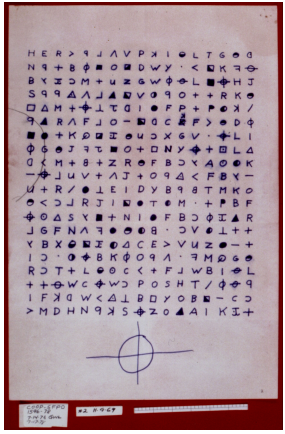
Breaking The Zodiac Killer's 340-character cipher



340 characters.
Unsolved for 51 years.

Oranchak, Blake, and Van Eycke (2024).
Read the paper: arXiv:2403.17350

Breaking The Zodiac Killer's 340-character cipher



340 characters.
Unsolved for 51 years.

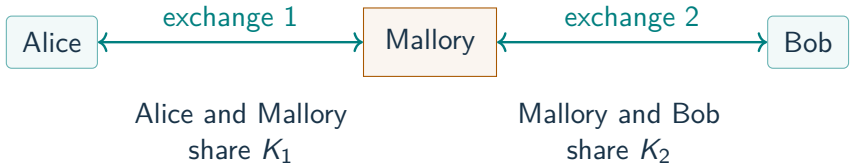
More than a simple substitution:

- One letter could have several symbols.
- The characters were also rearranged.

Oranchak, Blake, and Van Eycke (2024).
Read the paper: arXiv:2403.17350

Optional reading: Man-in-the-middle attacks

Basic DH and RSA encryption do not, by themselves, authenticate the other party.



Authentication: verify who is at the other end of the exchange.

Optional reading: Preventing impersonation

Authentication

Verify who owns the key, and authenticate the exchange.

Public-key infrastructure (PKI)
A trusted authority certifies whose public key it is; digital signatures authenticate the exchange.

A pre-shared secret
Use an already shared secret to authenticate the exchanged messages.

A trusted channel
Check public-key fingerprints in person, or use keys securely installed in advance.